

浙江大学

本科实验报告

课程名称:	课程综合实践 I- 深度学习基础——理论与实践
姓 名:	李昊泽
学 院:	计算机科学与技术学院
系:	
专 业:	计算机科学与技术
学 号:	3250102780
指导教师:	王总辉

2026 年 9 月 2 日

浙江大学实验报告

课程名称: 课程综合实践 I- 深度学习基础——理论与实践 实验类型: 综合

实验项目名称: 实验三: 卷积神经网络

学生姓名: 李昊泽 专业: 计算机科学与技术 学号: 3250102780

实验地点: 紫金港海洋大楼 11 机房 实验日期: 2026 年 9 月 2 日 指导老师: 王总辉

摘要

本实验使用卷积神经网络完成两个图像分类任务。第一部分在 Kaggle 平台完成 48×48 灰度人脸表情七分类。模型由三组卷积、批归一化、RReLU 与最大池化构成, 并使用分层验证、数据增强、类别加权交叉熵、AdamW、学习率调度和测试时增强。模型在 3300 个验证样本上的最佳准确率为 57.48%, 在 Kaggle 测试集上的 Accuracy Score 为 0.58679。

第二部分完成美国手语图像 24 分类。模型将 28×28 灰度图像依次变换为 25、50、75 通道的特征图, 再经全连接层输出类别 logits。20 轮训练中, 训练准确率最高达到 100.00%, 验证准确率最高达到 98.30%。

实验原理

一、CNN 与全连接网络

卷积神经网络使用较小的卷积核在图像上滑动, 同一卷积核在不同位置共享参数, 能够学习边缘、纹理、局部轮廓及更高层语义特征。

对于二维输入特征图 X , 第 k 个卷积核在位置 (i, j) 的输出可写为:

$$Y_{k,i,j} = b_k + \sum_c \sum_u \sum_v W_{k,c,u,v} X_{c,i+u,j+v}$$

卷积输出尺寸由输入尺寸 H_{in} 、卷积核大小 K 、步长 S 和填充 P 决定:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$$

本实验使用 3×3 卷积核、步长 1 和填充 1, 因此卷积前后的空间尺寸保持不变; 随后由 2×2 、步长 2 的最大池化将高和宽减半。

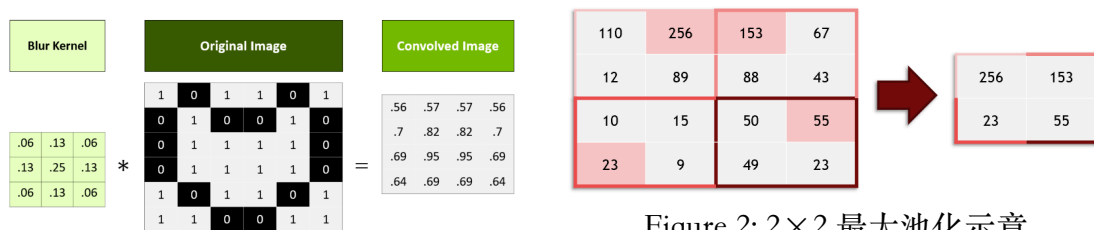


Figure 2: 2×2 最大池化示意

Figure 1: 卷积核在局部区域上提取特征

二、激活、归一化

Kaggle 表情模型使用 RReLU。它在负半轴保留一个随机小斜率，避免负输入的梯度恒为 0。模型输出层不添加 Softmax，而是直接给出 logits，因为 `CrossEntropyLoss` 已在内部完成 LogSoftmax 与负对数似然计算。

BatchNorm 对一个 mini-batch 中同一通道的激活进行标准化，并学习缩放和平移参数，使不同层的数值尺度更稳定。Dropout 在训练阶段随机将一部分激活置零，迫使网络不依赖少数神经元；在 `model.eval()` 下会自动关闭。

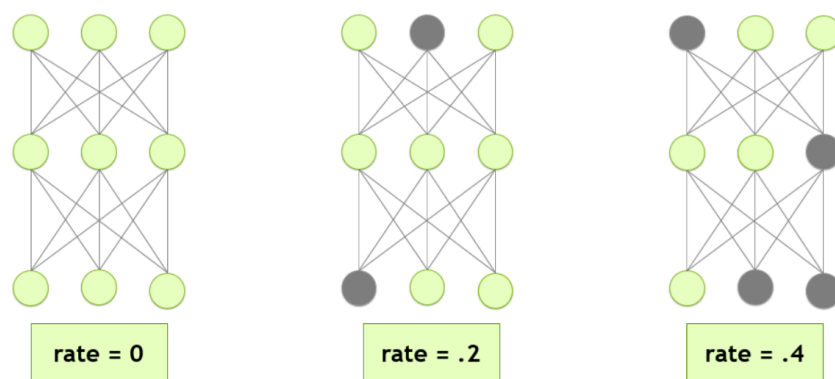


Figure 3: 不同 Dropout 比例下的随机失活示意

三、损失函数

设输出 logits 为 $z_0, z_1, \dots, z_{C-1}$ ，Softmax 概率为：

$$p_k = \frac{\exp(z_k)}{\sum_{j=0}^{C-1} \exp(z_j)}$$

真实类别为 y 时，单个样本的交叉熵损失为：

$$L = -\log p_y$$

实验一：Kaggle 表情图片分类

一、任务与数据

本任务将人脸表情划分为七类：0 生气、1 厌恶、2 恐惧、3 高兴、4 难过、5 惊讶、6 中立。每张图像以一个包含 2304 个灰度值的字符串存储在 `feature` 列中，对应 48×48 像素。

训练标签分布：

类别	数量	类别	数量	类别	数量	类别	数量
0	3062	1	355	2	3188	3	5529
4	3692	5	2375	6	3797		

Note

类别 1 只有 355 个样本，而类别 3 有 5529 个样本，类别分布差异明显。因此训练时采用类别加权交叉熵，避免模型只偏向样本较多的类别。

二、 数据预处理

使用空格分隔 `feature` 中的像素值，检查每行均含 2304 个数，再重塑为 `(N,1,48,48)` 的 PyTorch 张量。像素除以 255 后映射到 0~1，随后用均值 0.5、标准差 0.5 归一化到约 -1~1。

固定随机种子 42，并按标签分层抽取 15% 作为验证集，得到 18698 个训练样本、3300 个验证样本和 6711 个测试样本。

三、 网络结构

模型使用三段 `Conv2d → BatchNorm2d → RReLU → MaxPool2d`。

阶段	输入与输出	主要参数
卷积块 1	<code>1×48×48 → 64×24×24</code>	<code>3×3, P=1</code>
卷积块 2	<code>64×24×24 → 128×12×12</code>	<code>3×3, P=1</code>
卷积块 3	<code>128×12×12 → 256×6×6</code>	<code>3×3, P=1</code>
Flatten	<code>256×6×6 → 9216</code>	-
全连接层	<code>9216 → 512 → 128 → 7</code>	Dropout 0.3/0.5

该模型共有 5,156,807 个可训练参数。课程资料中的参考结构使用更宽的 4096、1024 和 256 维全连接层；本实验缩小全连接层，以减少参数量和过拟合风险，同时保留三段卷积主干。

```

ExpressionCNN(
  (features): Sequential(
    (0): ConvBlock(
      (0): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): RReLU(lower=0.125, upper=0.3333333333333333, inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (1): ConvBlock(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): RReLU(lower=0.125, upper=0.3333333333333333, inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (2): ConvBlock(
      (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): RReLU(lower=0.125, upper=0.3333333333333333, inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Dropout(p=0.3, inplace=False)
    (2): Linear(in_features=9216, out_features=512, bias=True)
    ...
  )
)
可训练参数: 5,156,807
形状检查: (2, 7)

```

Figure 4: 模型结构

四、训练策略

Kaggle 实验使用 Tesla T4 和 PyTorch 2.10.0+cu128，批大小为 256，共训练 30 个 epoch。损失函数为带类别权重和 0.05 标签平滑的交叉熵；优化器为 AdamW，初始学习率 0.001，权重衰减为 10^{-4} 。当验证准确率连续两轮没有改善时，学习率减半；同时设置梯度裁剪、混合精度训练和早停，并按照最高验证准确率保存 `best_expression_cnn.pth`。

五、实验结果与分析

选取训练过程中的代表性轮次如下：

Epoch	训练 Loss	训练 Acc	验证 Loss	验证 Acc
1	2.1937	19.94%	1.8974	35.48%
5	1.7367	36.65%	1.6534	43.79%
10	1.6055	44.37%	1.5559	49.15%
17	1.5037	49.05%	1.4850	54.09%
23	1.4270	52.06%	1.4301	56.12%
27	1.4010	53.53%	1.4139	57.18%
29	1.3982	53.59%	1.4158	57.48%
30	1.3910	54.48%	1.4036	56.73%

```

[01/30] 7.7s train loss=2.1937 acc=0.1994 | val loss=1.8974 acc=0.3548 | lr=1.00e-03 <- saved
[02/30] 6.9s train loss=1.9356 acc=0.2647 | val loss=1.8177 acc=0.2379 | lr=1.00e-03
[03/30] 6.7s train loss=1.8554 acc=0.3000 | val loss=1.7148 acc=0.4221 | lr=1.00e-03 <- saved
[04/30] 7.0s train loss=1.7926 acc=0.3340 | val loss=1.7166 acc=0.3879 | lr=1.00e-03
[05/30] 6.9s train loss=1.7367 acc=0.3665 | val loss=1.6534 acc=0.4379 | lr=1.00e-03 <- saved
[06/30] 6.6s train loss=1.7159 acc=0.3835 | val loss=1.6683 acc=0.4015 | lr=1.00e-03
[07/30] 6.6s train loss=1.6841 acc=0.3943 | val loss=1.5886 acc=0.4664 | lr=1.00e-03 <- saved
[08/30] 6.7s train loss=1.6535 acc=0.4120 | val loss=1.5883 acc=0.4867 | lr=1.00e-03 <- saved
[09/30] 6.6s train loss=1.6258 acc=0.4297 | val loss=1.5664 acc=0.4621 | lr=1.00e-03
[10/30] 6.7s train loss=1.6055 acc=0.4437 | val loss=1.5559 acc=0.4915 | lr=1.00e-03 <- saved
[11/30] 6.8s train loss=1.5942 acc=0.4463 | val loss=1.5227 acc=0.5061 | lr=1.00e-03 <- saved
[12/30] 6.8s train loss=1.5789 acc=0.4594 | val loss=1.5200 acc=0.4997 | lr=1.00e-03
[13/30] 6.5s train loss=1.5552 acc=0.4602 | val loss=1.5017 acc=0.4979 | lr=1.00e-03
[14/30] 6.6s train loss=1.5518 acc=0.4707 | val loss=1.5023 acc=0.5124 | lr=1.00e-03 <- saved
[15/30] 6.7s train loss=1.5269 acc=0.4781 | val loss=1.4937 acc=0.5239 | lr=1.00e-03 <- saved
[16/30] 6.6s train loss=1.5312 acc=0.4815 | val loss=1.4812 acc=0.5288 | lr=1.00e-03 <- saved
[17/30] 6.7s train loss=1.5037 acc=0.4905 | val loss=1.4850 acc=0.5409 | lr=1.00e-03 <- saved
[18/30] 6.7s train loss=1.5087 acc=0.4891 | val loss=1.4712 acc=0.5367 | lr=1.00e-03
[19/30] 6.7s train loss=1.4864 acc=0.4996 | val loss=1.4772 acc=0.5327 | lr=1.00e-03
[20/30] 6.6s train loss=1.4773 acc=0.5013 | val loss=1.4755 acc=0.5345 | lr=5.00e-04
[21/30] 6.9s train loss=1.4489 acc=0.5184 | val loss=1.4314 acc=0.5576 | lr=5.00e-04 <- saved
[22/30] 6.8s train loss=1.4388 acc=0.5210 | val loss=1.4418 acc=0.5524 | lr=5.00e-04
[23/30] 6.8s train loss=1.4270 acc=0.5206 | val loss=1.4301 acc=0.5612 | lr=5.00e-04 <- saved
[24/30] 6.6s train loss=1.4228 acc=0.5274 | val loss=1.4226 acc=0.5603 | lr=5.00e-04
[25/30] 6.6s train loss=1.4144 acc=0.5311 | val loss=1.4621 acc=0.5512 | lr=5.00e-04
...
[29/30] 6.7s train loss=1.3982 acc=0.5359 | val loss=1.4158 acc=0.5748 | lr=5.00e-04 <- saved
[30/30] 6.5s train loss=1.3910 acc=0.5448 | val loss=1.4036 acc=0.5673 | lr=5.00e-04

```

Figure 5: 训练结果

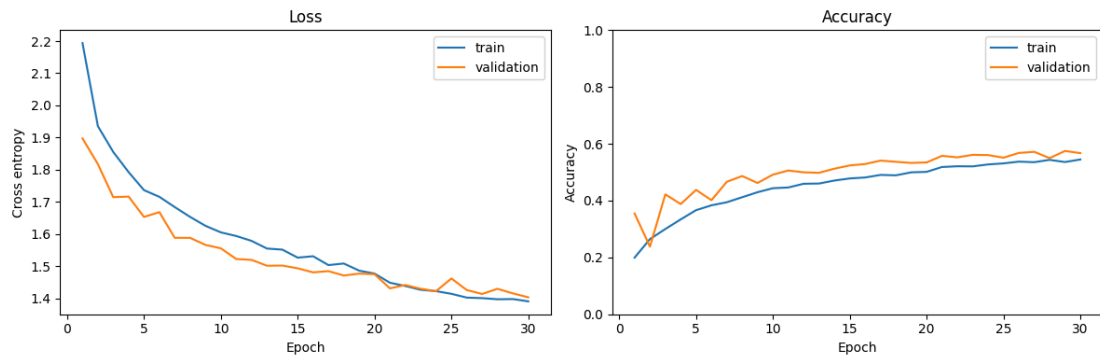


Figure 6: 训练曲线

最佳模型在验证集上的分类结果如下：

类别	Precision	Recall	F1	Support
Angry	0.5501	0.4423	0.4903	459
Disgust	0.1780	0.6415	0.2787	53
Fear	0.4681	0.3536	0.4029	478
Happy	0.7966	0.7976	0.7971	830
Sad	0.4874	0.4206	0.4516	554
Surprise	0.6532	0.7247	0.6871	356
Neutral	0.5007	0.5930	0.5430	570

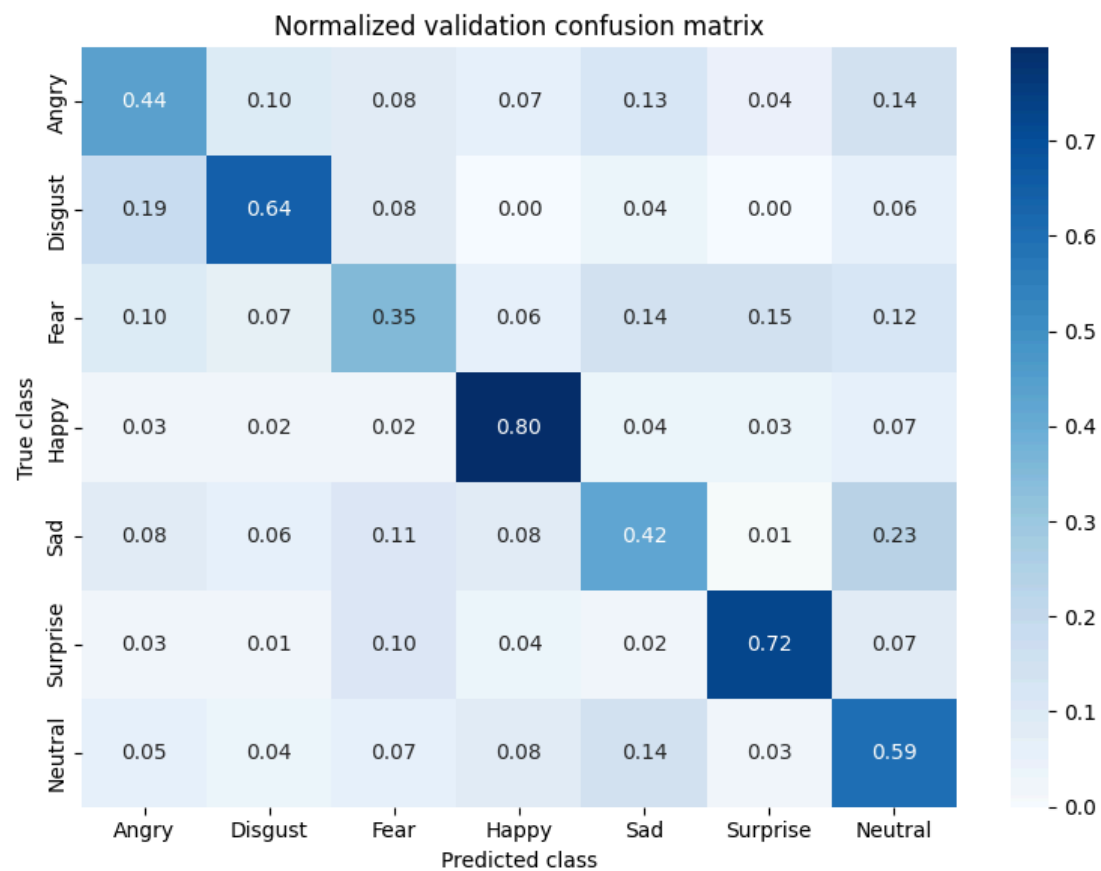


Figure 7: 混淆矩阵

Happy 的 F1 为 0.7971，是识别效果最好的类别；Surprise 次之。Disgust 只有 53 个验证样本，类别加权使其召回率达到 0.6415，但精确率仅为 0.1780，说明模型会把一部分其他表情误判为 Disgust。Fear 和 Sad 的 F1 也较低，反映出这些表情在低分辨率灰度图片中的区分难度较高。

最终 Accuracy Score 为 0.58679，即约 58.68%。

实验二：美国手语图片识别（卷积 version）

一、任务

本任务的数据处理与 lab2 基本相同：每张图像为 28×28 的单通道灰度图像，CSV 的第一列为 `label`，其余 784 列为像素。J 和 Z 需要连续运动才能表达，因此静态数据集中不包含这两个字母；本地数据已将其余类别连续映射为 0 ~ 23，共 24 类。

二、网络结构

ASL 模型包含三层卷积，每层使用 3×3 卷积、步长 1、填充 1，卷积后接 BatchNorm 和 ReLU。池化将图像尺寸从 28×28 依次减半到 14×14 、 7×7 和 3×3 。第二个卷积块加入 0.2 Dropout，全连接层前加入 0.3 Dropout。

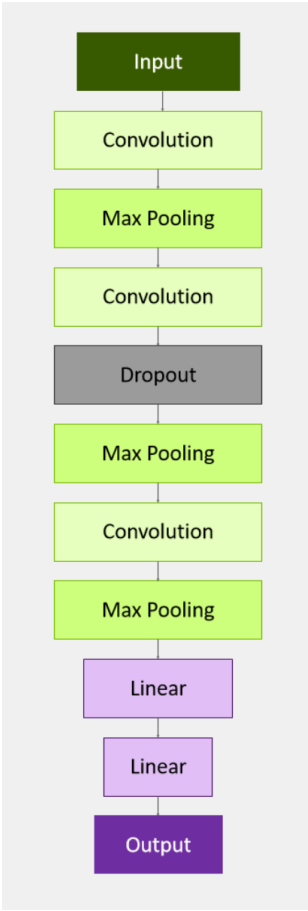


Figure 8: ASL CNN 整体结构

阶段	输入与输出	层配置	作用
卷积块 1	$1 \times 28 \times 28 \rightarrow 25 \times 14 \times 14$	Conv + BN + ReLU + Pool	提取基础边缘特征
卷积块 2	$25 \times 14 \times 14 \rightarrow 50 \times 7 \times 7$	Conv + BN + ReLU + Dropout + Pool	组合局部手指轮廓
卷积块 3	$50 \times 7 \times 7 \rightarrow 75 \times 3 \times 3$	Conv + BN + ReLU + Pool	形成更高层手势特征
Flatten	$75 \times 3 \times 3 \rightarrow 675$	-	转为一维特征向量
全连接	$675 \rightarrow 512 \rightarrow 24$	Dropout 0.3	输出 24 类 logits

该网络共有 404099 个可训练参数。与上一实验的全连接 ASL 模型相比，它在展平前保留二维空间关系，更适合识别手指边缘、局部轮廓和不同手势的相对位置。

```

Sequential(
  (0): Conv2d(1, 25, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(25, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU()
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(25, 50, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(50, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU()
  (7): Dropout(p=0.2, inplace=False)
  (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (9): Conv2d(50, 75, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (10): BatchNorm2d(75, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (11): ReLU()
  (12): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (13): Flatten(start_dim=1, end_dim=-1)
  (14): Linear(in_features=675, out_features=512, bias=True)
  (15): Dropout(p=0.3, inplace=False)
  (16): ReLU()
  (17): Linear(in_features=512, out_features=24, bias=True)
)

```

Figure 9: 模型结构

三、训练与验证

模型使用 `CrossEntropyLoss` 和 Adam 默认参数训练 20 个 epoch。训练函数在 `model.train()` 下依次执行前向传播、清空梯度、计算损失、反向传播和参数更新；验证函数在 `model.eval()` 与 `torch.no_grad()` 下只进行前向传播。

```

1  def train():
2      model.train()
3      for x, y in train_loader:
4          output = model(x)
5          optimizer.zero_grad()
6          batch_loss = loss_function(output, y)
7          batch_loss.backward()
8          optimizer.step()
9
10 def validate():
11     model.eval()
12     with torch.no_grad():
13         for x, y in valid_loader:
14             output = model(x)

```

四、实验结果与分析

选取代表性训练结果如下。Notebook 中 epoch 从 0 开始编号：

Epoch	训练 Loss	训练 Acc	验证 Loss	验证 Acc
0	268.6069	90.74%	25.1972	95.71%
1	16.9970	99.49%	19.8946	97.24%
2	12.3731	99.65%	11.4613	97.91%
7	1.4564	99.96%	17.0607	97.84%
12	9.1179	99.72%	10.9429	98.09%
15	0.3725	99.99%	14.1010	98.30%
16	0.0547	100.00%	12.5894	98.28%
17	0.0452	100.00%	16.7879	98.26%
18	0.1935	99.99%	400.2440	73.70%
19	14.9442	99.54%	257.0182	83.41%

验证准确率在 Epoch 15 达到最高值 98.30%，Epoch 16 和 17 仍保持在 98.2% 左右；但 Epoch 18 突然降至 73.70%，最后一轮仅恢复到 83.41%。同时训练准确率几乎一直保持在 100%，说明模型已经充分拟合训练数据，而验证表现存在明显波动。

从现有证据可以确认：CNN 相比上一实验的全连接模型显著提高了 ASL 验证准确率，证明保留二维局部结构对手势识别有效。

参考资料

1. 课程资料
2. Kaggle Competition: 2024ZJUTEST3, Deep Learning class Experiment 3。
3. NVIDIA Deep Learning Institute, [03_asl_cnn.ipynb](#) 美国手语 CNN 实验。
4. 本报告的部分文字组织、代码核对与排错分析由 OpenAI Codex 辅助完成。所有实验数据、运行输出和技术描述均由本人检查，最终报告由本人审阅并负责。